BANKING MANAGEMENT SYSTEM

Project Report

Design, implementation, synchronization, and evaluation overview

Implementation-specific documentation

Important: Features and limitations are documented from the project source files and README. Tests are not claimed as independently executed for this documentation package.

Executive summary

This project is a terminal-based banking management system written in C. A client communicates with a TCP server on port 8080. The server forks a child process for each request, validates the stored user role and session state, and performs operations against fixed-width records in plain-text files. The storage layer uses fcntl byte-range locks for record-level coordination and fsync() after writes. A stress-test program issues concurrent requests to test one-session login enforcement and balance invariants.

1. Objectives and scope

- Implement customer, bank employee, manager, and administrator workflows over a client-server interface.
- Demonstrate POSIX sockets, process-per-request concurrency, file I/O, record locking, and synchronization.
- Provide persistent records for users, accounts, transactions, loans, and feedback without a relational database.
- Demonstrate financial invariants and concurrent-operation checks via stress_test.c.

The application is a teaching/demo system, not a production banking platform. In particular, the source documents plain-text passwords, floating-point balances, and manual rollback for transfers as known limitations.

2. Role functionality

Role / ID	Implemented capabilities
Customer / 1	Login and self-registration; view balance; deposit; withdraw; transfer; view transaction history; apply for loan and view own loans; submit feedback; change password; logout/exit.
Bank Employee / 2	Add customer with opening balance; modify customer by resetting password; view a customer passbook; view assigned loans; approve/reject an assigned loan; change own password.
Manager / 3	Activate/deactivate customer users; view loans and employees; assign loan applications to employees; review feedback; change own password.
Administrator / 4	List users; add employees; modify user details/reset password; manage roles; change own password.

3. Architecture

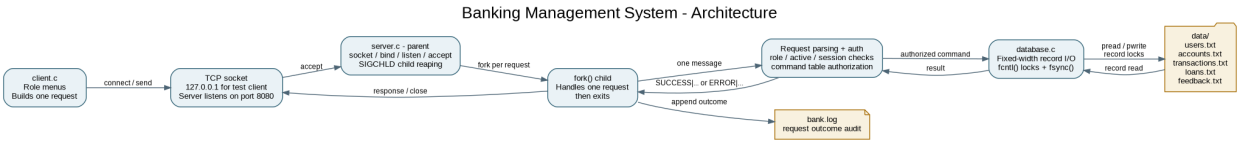


Figure 1. System architecture and file-backed persistence.

The server binds to all IPv4 interfaces (INADDR_ANY) on port 8080 with backlog 10. Each accepted request is handled by a forked child. The child opens its own data-file descriptors, processes one request, sends a SUCCESS|... or ERROR|... response, logs the outcome, closes the connection, and exits. The parent installs a SIGCHLD handler to reap terminated children.

4. Storage and record model

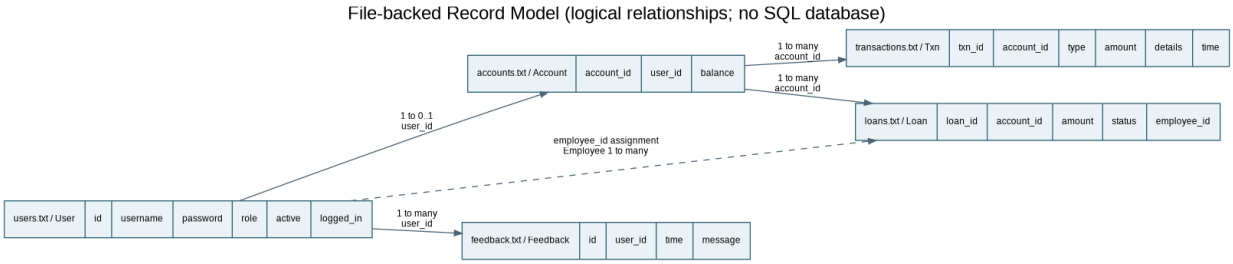


Figure 2. Logical relationships among the five record files. They are plain-text files, not SQL tables.

File	Record size	Field sequence	Purpose
data/users.txt	128 bytes	id | username | password | role | active | logged_in	Authentication, role, activation, session flag
data/accounts.txt	128 bytes	account_id | user_id | balance	Customer account and balance
data/transactions.txt	160 bytes	txn_id | account_id | type | amount | details | time	Passbook and operation ledger
data/loans.txt	128 bytes	loan_id | account_id | amount | status | employee_id	Loan state and assigned employee
data/feedback.txt	384 bytes	id | user_id | time | message	Customer feedback

Every record is padded to a fixed size and terminated by a newline. Because record N starts at offset N × record_size, the file layer can lock an individual record by locking its exact byte range. Append operations use a whole-file lock.

5. Concurrency and synchronization

- fork() isolates handling of separate requests into separate processes.
- fcntl(F_SETLKW) obtains blocking byte-range locks around records being modified.
- Handlers re-read protected records after taking a lock, reducing stale-read/lost-update problems.
- Transfers lock both account records in increasing record-index order to avoid circular wait for opposite-direction transfers.

- New user, loan, and transaction records are appended while the corresponding table is locked.
- rec_write() uses pwrite() followed by fsync() to request durable persistence of each record write.
- SIGCHLD handling calls waitpid(-1, NULL, WNOHANG) to reap terminated children.

6. Financial operations and consistency model

Deposits and withdrawals validate a positive amount and update the account record under a lock. Transfers lock the sender and recipient records, re-read both balances, check available funds, update both records, and attempt to restore the sender if the recipient write fails. Successful financial operations append transaction records. Loan applications begin as PENDING; the manager assigns them to an employee as ASSIGNED; an employee may APPROVE or REJECT. Approval credits the account and records a LOAN_CREDIT transaction. If the credit operation fails, the loan status is restored to ASSIGNED.

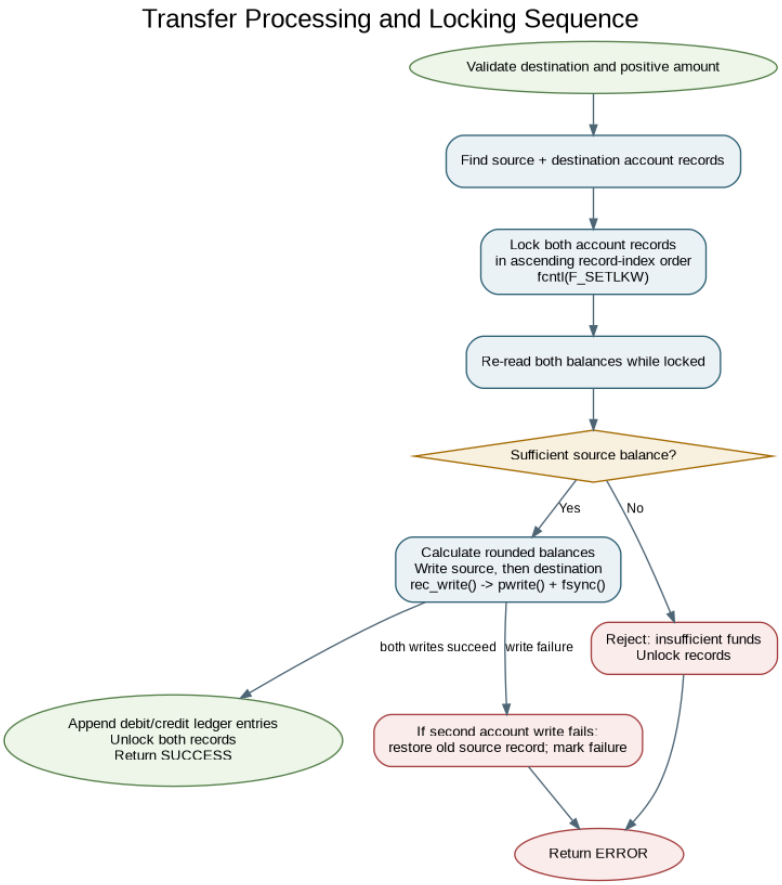


Figure 3. Transfer path and manual rollback behavior.

7. Testing and evidence

The project includes stress_test.c. Its described scenarios cover five simultaneous logins for a single employee account, twenty concurrent deposits, ten concurrent withdrawals from a constrained balance, and ten concurrent transfers. The README states an expected result of “6 passed, 0 failed.” This documentation package has not independently executed that test; the expected result should be verified on the submission machine and captured as evidence.

Test category	Evidence to capture
Functional	Role login/menu screenshots or terminal output; successful and rejected operation responses.
Concurrency	Full stress_test output, with server and test process running in separate terminals.
Storage	Before/after contents of data/accounts.txt and corresponding transaction entries.
Session behavior	Second simultaneous login rejected; re-login succeeds after logout.
Failure behavior	Insufficient-funds result; invalid role/loan ownership attempt; transfer write-failure behavior if testable.

8. Limitations and risks

- Passwords are stored as plaintext in users.txt. This is unsafe outside a classroom demo.
- Balances and amounts use double. Floating-point representation is not an ideal monetary representation.
- Transfer atomicity is implemented manually by writing the old sender record back if the receiver write fails; a process crash between writes can still leave an inconsistent transfer. Transaction-ledger append failures are not propagated through txn_add(), so a balance update may be reported successful even if its ledger write fails.
- The protocol uses a pipe-delimited format and a single recv() for each request, so delimiter handling and TCP message framing should be treated as constraints.
- Records are found using linear scans, so lookup cost grows with the number of records.
- The server binds to INADDR_ANY without transport encryption or an authenticated network channel; do not expose it to untrusted networks.

Conclusion

The project makes low-level operating-system mechanisms visible and explainable: socket communication, process creation, file descriptors, record locking, persistence, and signal-based child reaping. Its tradeoff is that consistency and data integrity must be maintained by application code, and several security/durability limitations remain explicitly acknowledged.